

Homework 20: Build It Yourself

Build a real two-package project from scratch, package it, then reproduce three startup failures so you have seen each one deliberately.

Everything must happen in a temporary directory that is deleted before the program exits.

Part One: Two Packages

Create `ProjectBuilder.java` in the classic form.

Have it **write, compile and run** a small project with this layout:

```
src/com/example/model/Item.java
src/com/example/app/Inventory.java
```

`Item` is a record with a name and a quantity. It must have:

- A **public** method other packages call
- A **package-private** helper the public method uses

`Inventory` imports `Item`, builds a few, and prints them.

Compile with `javac` through `ToolProvider`, list the class files produced, then load and run

`Inventory` through a `URLClassLoader` pointed at the output directory.

Add a comment inside `Inventory` showing the line that would **not** compile if it tried to call the package-private helper, with the real error text. Check it.

Explain It

A comment on why the directory path had to mirror the package statement. Say what `javac` uses it for and what the JVM uses it for. They are not the same reason.

Part Two: Package It

Build a jar with `--main-class` set, then list its contents.

Also run `jar --describe-module` against it and print what comes back.

Explain It

Two comments:

1. **What `--main-class` actually did.** Name the file it wrote into and say what would still work without it.
2. **What `--describe-module` reported.** You did not write a `module-info.java`, so what did the tool derive, from where, and why is that not real modularity?

Part Three: Three Startup Failures

Reproduce each, print what you caught, and explain it:

1. **`ClassNotFoundException` for a class that never existed.** Say why the compiler could not have warned you, and name two real situations that cause this.
2. **`ClassNotFoundException` for a class that exists but is not on the loader's path.** Build an empty `URLClassLoader` and ask it for a class you just compiled.

Then write the interesting comment: the symptom is identical to failure 1 and the cause is completely different. Say what this one corresponds to in a real deployment.

3. **`NoClassDefFoundError` from a failed static initialiser.** Touch the class twice and print both results.

This is the one worth getting right. Write a comment covering:

- What the first attempt reports, and where the real cause is hiding
- Why the second attempt reports something less useful
- **The diagnostic rule.** What should you do when you see `NoClassDefFoundError` for a class you are certain is present?

Part Four: Modules In Practice

Print, with labels:

- `String`'s class loader, and what `null` means there
- Your own class's loader
- `String`'s module name
- Your own module, and whether it is named

Explain It

A comment on:

- What the **unnamed module** is, and what it reads and exports
- **Why `--add-opens` exists.** Be specific: what does `java.base` not do, what exception does a library get as a result, and what does the flag change?

Acceptance Criteria

- ☐ Runs with `java ProjectBuilder.java` and deletes its temp directory
- ☐ Two packages are written, compiled and run
- ☐ `Item` has both a public and a package-private member
- ☐ The non-compiling call is documented with its real error text
- ☐ A comment explains the directory rule from both `javac`'s and the JVM's side
- ☐ A jar is built with `--main-class` and its contents listed
- ☐ `--describe-module` output is printed and explained
- ☐ All three failures are reproduced with their real messages
- ☐ Failures 1 and 2 are distinguished by cause, not symptom
- ☐ The static-initialiser case shows both attempts
- ☐ A diagnostic rule for `NoClassDefFoundError` is stated
- ☐ The unnamed module and `--add-opens` are both explained

Stretch

Add a real `module-info.java` that exports only `com.example.model` and **not** `com.example.app`. Compile it as a module and run it with `--module-path`.

Then try to reflect into the unexported package from outside and record what you get.

Write a comment on what strong encapsulation gave you that the jar in part two did not, and what it cost in exchange.

Hint, if failure 2 refuses to fail

An ordinary `new URLClassLoader(urls)` delegates to its parent first, and the parent can probably find your class. Pass `null` as the parent to stop the delegation, and then the loader really does have nowhere to look.